User Manual

for S32K1_S32M24X CANTRCV_43_AE Driver

Document Number: UM2CANTRCV_43_AEASRR21-11 Rev0000R3.0.0 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

This User Manual describes NXP Semiconductors' AUTOSAR CanTrcv Driver for S32M24x.

AUTOSAR CanTrcv Driver configuration parameters description can be found in the Tresos Configuration Plugin section. Deviations from the specification are described in the Deviations from Requirements section.

AUTOSAR CanTrcv driver requirements and APIs are described in the CanTrcv Driver Software Specification Document (version R21-11).

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".

- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Default Error Tracer
MCU	Micro controller Unit
N/A	Not Available
CANTRCV	Can Transceiver
AE	Application Extension
CAN	Controller Area Network

- The term "CanTrcv Driver" is related to the software handling the CANPHY channel.
- The term "Application" is used for the software utilizing the CanTrcv Driver.

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR R21-11 CanTrcv Driver Software Specification document (See Table Reference List)

3.2 Driver Design Summary

The CanTrcv Driver controls the CANPHY in AE of the S32M24x devices. It provides the following features:

- Configuration and initialization of the CANPHY.
- Support Normal and Standby modes.
- Support Wake Up Pattern (WUP) in Standby mode.

3.3 Hardware Resources

Driver has one CANPHY channel in AE which is a companion die that is assembled along with an MCU die.

Note

The user must select properly the chip which supports CanPHY: P32M244CC, P32M242CC

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR CANTRCV Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the CANTRCV Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the CANTRCV driver.

Requirement	Status	Description	Notes
SWS_CanTrcv_00230	N/S	The CAN Tranceiver Driver shall use the Time service Tm_Busy↩ Wait1us16bit to realize the wait time for transceiver state changes.	Applicable only for platforms which need timeout monitors
SWS_CanTrcv_00174	N/S	If selective wakeup is supported by the transceiver hardware, it shall be indicated with the configuration parameter CanTrcvHwPnSupport.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00175	N/S	The configuration container for selective wakeup functionality (Can↩ TrcvPartialNetwork) and for the following APIs: - 8.4.7 Can↩ Trcv_GetTrcvSystemData, - 8.↩ 4.8 CanTrcv_ClearTrcvWufFlag, - 8.4.9 CanTrcv_ReadTrcvTimeout↩ Flag, - 8.4.10 CanTrcv_ClearTrcv↩ TimeoutFlag and - 8.4.11 Can↩ Trcv_ReadTrcvSilenceFlag shall exist only if CanTrcvHwPnSupport = TRUE.	Applicable only for platforms which support Selective wakeup feature The CanTrcvPartialNetwork/Can↩ TrcvBusErrFlag is still configuable to enable/disable CanTrcv_Main↩ FunctionDiagnostics schedule function
SWS_CanTrcv_00177	N/S	If selective wakeup is supported, CAN transceivers shall be configured to wake up on a particular CAN frame or a group of CAN frames using the parameters Can↩ TrcvPnFrameCanId, CanTrcvPn↩ FrameCanIdMask and CanTrcv↩ PnFrameDataMask.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00181	N/S	If selective wakeup is enabled and supported by hardware: POR and SYSERR flags of the transceiver status shall be checked by CanTrcv_↩ Init API.	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00182	N/S	If the POR flag or SYSERR flag is set, transceiver shall be reconfigured for selective wakeup functionality by running the configuration sequence. If the POR flag or SYSERR flag is not set, the configuration stored in the transceiver memory will be still valid and reconfiguration is not necessary.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00183	N/S	If the POR flag is set, wakeup shall be reported to EcuM through API EcuM_SetWakeupEvent with a wakeup source value, which has a "1" at the bit position according to the symbolic name value referred by CanTrcvPorWakeupsourceRef, and "0" on all others.	Applicable only for platforms which support Selective wakeup feature refer SWS_CanTrcv_00181
SWS_CanTrcv_00184	N/S	If the SYSERR flag is set, wakeup shall be reported to EcuM through API EcuM_SetWakeupEvent with a wakeup source value, which has a "1" at the bit position according to the symbolic name value referred by CanTrcvSyserrWakeupSourceRef, and "0" on all others.	Applicable only for platforms which support Selective wakeup feature refer SWS_CanTrcv_00181
SWS_CanTrcv_00168	N/S	If development error detection is enabled for CanTrcv module: the function CanTrcv_Init shall raise the development error CANTRCV_E_BAUDRATE_NOT_SUPPORTED, if the configured baud rate is not supported by the transceiver.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00226	N/S	In order to implement the AUTOSAR Partial Networking mechanism CAN transceivers shall support the definition of a data mask for the Wake Up Frame (the configuration structure of CanTrcvPnFrameDataMask is mandatory).	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00186	N/S	If selective wakeup is supported by hardware: the flags POR and SYSERR of the transceiver status shall be checked by CanTrcv_SetOpMode API.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00187	N/S	If the POR flag is set, transceiver shall be re-initialized to run the transceiver's configuration sequence.	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00188	N/S	If the SYSERR flag is NOT set and the requested mode is CANT↩ RCV_NORMAL, transceiver shall call the API CanIf_ConfirmPn↩ Availability() for the corresponding abstract CanIf Transceiver↩ Id. CanIf_ConfirmPnAvailability informs CanNm (through CanIf and CanSm) that selective wakeup is enabled.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00189	N/S	The function CanTrcv_Get↩ TrcvSystemData shall read the configuration/status of the CAN transceiver and store the read data in the out parameter TrcvSysData. If this is successful, E_OK shall be returned. Hint: This API can be invoked through diagnostic services or during initialization to determine the transceiver status and its availability. Note: Currently an agreement on the parameter set for the transceiver HW specification has not been reached. For this reason, the diagnostic data is now returned as a uint32 (as stored in the transceiver registers). When a definitive and standard parameter set is defined, a data structure may be defined for abstracting the diagnostic data.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00190	N/S	If there is no/incorrect communication to the transceiver, the function CanTrcv_GetTrcvSystemData shall report the runtime error code CANTRCV_E_NO_TRCV↩ _CONTROL to the default Error Tracer and return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00191	N/S	If development error detection is enabled for the CanTrcv module: if called before the CanTrcv has been initialized, the function CanTrcv_↩ GetTrcvSystemData shall raise development error CANTRCV_E_↩ UNINIT otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00192	N/S	If development error detection is enabled for the CanTrcv module: if called with an invalid transceiver ID for parameter Transceiver, function CanTrcv_GetTrcvSystemData shall raise the development error CANTRCV_E_INVALID_TRANSCEIVER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00193	N/S	If development error detection is enabled for the CanTrcv module: if called with NULL pointer for parameter TrcvSysData, function CanTrcv_GetTrcvSystemData shall raise the development error CANTRCV_E_PARAM_POINTER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00194	N/S	The function CanTrcv_ClearTrcvWufFlag shall clear the wakeup flag in the CAN transceiver. If successful, E_OK shall be returned. Implementation Hints: This API shall be used by the CanSM module for ensuring that no frame wakeup event is lost, during entering a low-power mode. This API clears the WUF flag. The CAN transceiver shall be shall be put into Standby mode (CANTRCV_STANDBY) after clearing of the WUF flag. If a system error (SYSERR, e.g. configuration error) occurs while selective wakeup functionality is being enabled, transceiver will disable the functionality. Transceiver will wake up on the next CAN wake pattern (WUP). In case of any other hardware error (e.g. frame detection error), transceiver will wake up if the error counter inside the transceiver overflows.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00195	N/S	CanTrcv shall inform CanIf that the wakeup flag has been cleared for the requested Transceiver, through the callback notification CanIf_ClearTrcvWufFlagIndication referring to the corresponding CAN transceiver with the abstract CanIf TransceiverId.	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00196	N/S	If there is no/incorrect communication to the transceiver, the function CanTrcv_ClearTrcvWuf↩ Flag shall report the runtime error CANTRCV_E_NO_TRCV↩ _CONTROL to the Default Error Tracer and return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00197	N/S	If development error detection is enabled for the CanTrcv module: if called before the CanTrcv has been initialized, the function CanTrcv_↩ ClearTrcvWufFlag shall raise development error CANTRCV_E_UN↩ INIT otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00198	N/S	If development error detection is enabled for the CanTrcv module: if called with an invalid transceiver ID for parameter Transceiver, function CanTrcv_ClearTrcvWufFlag shall raise the development error CANTRCV_E_INVALID_TRA↩ NSCEIVER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00199	N/S	If development error detection is enabled for the module Can↩ Trcv: If called with an invalid transceiver ID Transceiver, the function CanTrcv_ReadTrcvTimeout↩ Flag shall raise the development error CANTRCV_E_INVALID_T↩ RANSCEIVER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00200	N/S	If development error detection is enabled for the module CanTrcv↩ : If called with FlagState = NULL, the function CanTrcv_ReadTrcv↩ TimeoutFlag shall raise the development error CANTRCV_E_PAR↩ AM_POINTER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00201	N/S	If development error detection is enabled for the module Can↩ Trcv: If called with an invalid transceiver ID Transceiver, the function CanTrcv_ClearTrcvTimeout↩ Flag shall raise the development error CANTRCV_E_INVALID_T↩ RANSCEIVER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00202	N/S	If development error detection is enabled for the module CanTrcv: If called with an invalid transceiver ID Transceiver, the function CanTrcv_ReadTrcvSilenceFlag shall raise the development error CANTRCV_E_INVALID_TRANSCEIVER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00203	N/S	If development error detection is enabled for the module CanTrcv: If called with FlagState = NULL, the function CanTrcv_ReadTrcvSilenceFlag shall raise the development error CANTRCV_E_PARAM_POINTER otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00220	N/S	If development error detection for the module CanTrcv is enabled: If called before the CanTrcv module has been initialized, the function CanTrcv_SetPNActivationState shall raise the development error CANTRCV_E_UNINIT otherwise (if DET is disabled) return E_NOT_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00221	N/S	CanTrcv shall enable the PN wakeup functionality when function CanTrcv_SetPNActivationState is called with ActivationState= PN_ENABLED and return E_OK.	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00222	N/S	CanTrcv shall disable the PN wakeup functionality when function CanTrcv_SetPNActivationState is called with ActivationState= PN_DISABLED and return E_OK.	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00216	N/S	Service Name: CanTrcv_Clear↩ TrcvTimeoutFlag Syntax: Std_↩ ReturnType CanTrcv_ClearTrcv↩ TimeoutFlag (uint8 Transceiver) Service ID [hex]: 0x0c Sync/↩ Async: Synchronous Reentrancy↩ : Non Reentrant Parameters (in)↩ : Transceiver: CAN transceiver ID. Parameters (inout): None Parameters (out): None Return value↩ : Std_ReturnType: E_OK: Will be returned, if the timeout flag is successfully cleared. E_NOT_OK↩ : Will be returned, if the timeout flag could not be cleared. Description: Clears the status of the timeout flag in the transceiver hardware. This API shall exist only if CanTrcv↩ HwPnSupport = TRUE. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00214	N/S	Service Name: CanTrcv_Clear↩ TrcvWufFlag Syntax: Std_↩ ReturnType CanTrcv_Clear↩ TrcvWufFlag (uint8 Transceiver) Service ID [hex]: 0x0a Sync/↩ Async: Synchronous Reentrancy: Reentrant for different transceivers Parameters (in): Transceiver: C↩ AN Transceiver ID. Parameters (inout): None Parameters (out): None Return value: Std_Return↩ Type: E_OK: will be returned if the WUF flag has been cleared. E_NOT_OK: will be returned if the WUF flag has not been cleared or a development error occurs. Description: Clears the WUF flag in the transceiver hardware. This API shall exist only if CanTrcv↩ HwPnSupport = TRUE. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00213	N/S	Service Name: CanTrcv_GetTrcv↩ SystemData Syntax: Std_Return↩ Type CanTrcv_GetTrcvSystem↩ Data (uint8 Transceiver, uint32∗ TrcvSysData) Service ID [hex]: 0x09 Sync/Async: Synchronous Reentrancy: Non Reentrant Parameters (in): Transceiver↩ : CAN transceiver ID. Parameters (inout): None Parameters (out): TrcvSysData: Configuration/Status data of the transceiver. Return value: Std_ReturnType: E_OK: will be returned if the transceiver status is successfully read. E↩ _NOT_OK: will be returned if the transceiver status data is not available or a development error occurs. Description: Reads the transceiver configuration/status data and returns it through parameter TrcvSysData. This API shall exist only if CanTrcvHwPnSupport = TRUE. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00217	N/S	Service Name: CanTrcv_Read↩ TrcvSilenceFlag Syntax: Std_↩ ReturnType CanTrcv_ReadTrcv↩ SilenceFlag (uint8 Transceiver, CanTrcv_TrcvFlagStateType∗ FlagState) Service ID [hex]↩ : 0x0d Sync/Async: Synchronous Reentrancy: Non Reentrant Parameters (in): Transceiver: C↩ AN transceiver ID. Parameters (inout): None Parameters (out): FlagState: State of the silence flag. Return value: Std_ReturnType: E_OK: Will be returned, if status of the silence flag is success-fully read. E_NOT_OK: Will be returned, if status of the silence flag could not be read. Description: Reads the status of the silence flag from the transceiver hardware. This API shall exist only if CanTrcvHwPn↩ Support = TRUE. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00215	N/S	Service Name: CanTrcv_Read↩ TrcvTimeoutFlag Syntax: Std_↩ ReturnType CanTrcv_ReadTrcv↩ TimeoutFlag (uint8 Transceiver, CanTrcv_TrcvFlagStateType∗ FlagState) Service ID [hex]↩ : 0x0b Sync/Async: Synchronous Reentrancy: Non Reentrant Parameters (in): Transceiver: C↩ AN transceiver ID. Parameters (inout): None Parameters (out): FlagState: State of the timeout flag. Return value: Std_Return↩ Type: E_OK: Will be returned, if status of the timeout flag is success-fully read. E_NOT_OK: Will be returned, if status of the timeout flag could not be read. Description: Reads the status of the timeout flag from the transceiver hardware. This API shall exist only if CanTrcvHwPnSupport = TRUE. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00219	N/S	Service Name: CanTrcv_SetP↩ NActivationState Syntax: Std↩ _ReturnType CanTrcv_SetPN↩ ActivationState (CanTrcv_P↩ NActivationType ActivationState) Service ID [hex]: 0x0f Sync/↩ Async: Synchronous Reentrancy: Non Reentrant Parameters (in): ActivationState: PN_ENABL↩ ED: PN wakeup functionality in CanTrcv shall be enabled. PN_D↩ IABLED: PN wakeup functionality in CanTrcv shall be disabled. Parameters (inout): None Parameters (out): None Return value: Std_ReturnType: E_OK: Will be returned, if the PN has been changed to the requested configuration. E_NOT_OK: Will be returned, if the PN configuration change has failed. The previous configuration has not been changed. Description: The API configures the wake-up of the transceiver for Standby and Sleep Mode: Either the CAN transceiver is woken up by a remote wake-up pattern (standard CAN wake-up) or by the configured remote wake-up frame. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
SWS_CanTrcv_00210	N/S	Name: CanTrcv_PNActivation↩ Type Kind: Enumeration Range↩ : PN_ENABLED: –: PN wakeup functionality in CanTrcv is enabled. PN_DISABLED: –: PN wakeup functionality in CanTrcv is disabled. Description: Datatype used for describing whether PN wakeup functionality in CanTrcv is enabled or disabled. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature
SWS_CanTrcv_00211	N/S	Name: CanTrcv_TrcvFlagState↩ Type Kind: Enumeration Range↩ : CANTRCV_FLAG_SET: –: The flag is set in the transceiver hardware. CANTRCV_FLAG_CLE↩ ARED: –: The flag is cleared in the transceiver hardware. Description↩ : Provides the state of a flag in the transceiver hardware. Available via: CanTrcv.h	Applicable only for platforms which support Selective wakeup feature
ECUC_CanTrcv_00097	N/S	Name: CanTrcvControlsPower↩ Supply Parent Container: Can↩ TrcvChannel Description: Is E↩ CU power supply controlled by this transceiver? TRUE = Controlled by transceiver. FALSE = Not controlled by transceiver. Multiplicity: 1 Type: Ecuc↩ BooleanParamDef Default value↩ : false Post-Build Variant Value↩ : false Value Configuration Class↩ : Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local	The CanTrcv Drvier will not control Power Supply as it's done in other Driver
ECUC_CanTrcv_00160	N/S	Name: CanTrcvHwPnSupport Parent Container: CanTrcvChannel Description: Indicates whether the HW supports the selective wakeup function: TRUE = Selective wakeup feature is supported by the transceiver FALSE = Selective wakeup functionality is not available in transceiver Multiplicity: 1 Type: EcucBooleanParamDef Default value: false Post-Build Variant Value: false Value Configuration Class: Pre-compile time: X↩ : All Variants Link time: – Post-build time: – Scope / Dependency↩ : scope: local: dependency: Can↩ TrcvWakeUpSupport	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
ECUC_CanTrcv_00181	N/S	Name: CanTrcvPorWakeup↩ SourceRef Parent Container: CanTrcvChannel Description↩ : Symbolic name reference to specify the wakeup sources that should be used in the calls to EcuM_SetWakeupEvent as specified in [SWS_CanTrcv_00183] and [SWS_CanTrcv_00184].↩ : This reference is mandatory if the HW supports POR or S↩ YSERR flags Multiplicity: 0..1 Type: Symbolic name reference to [EcuMWakeupSource] Post-↩ Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: ECU	Applicable only for platforms which support Selective wakeup feature
ECUC_CanTrcv_00182	N/S	Name: CanTrcvSyserrWakeup↩ SourceRef Parent Container: CanTrcvChannel Description↩ : Symbolic name reference to specify the wakeup sources that should be used in the calls to EcuM_SetWakeupEvent as specified in [SWS_CanTrcv_00183] and [SWS_CanTrcv_00184]↩ : This reference is mandatory if the HW supports POR or S↩ YSERR flags Multiplicity: 0..1 Type: Symbolic name reference to [EcuMWakeupSource] Post-↩ Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: ECU	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
ECUC_CanTrcv_00174	N/S	Name: CanTrcvSPICommTimeout Parent Container: CanTrcvConfig↩ Set Description: Indicates the maximum time allowed to the CanTrcv for replying (either positively or negatively) to a SPI command. Timeout is configured in milliseconds. Timeout value of '0' means that no specific timeout is to be used by CanTrcv and the communication is executed at the best of the SPI HW capacity. Multiplicity: 1 Type↩ : EcucIntegerParamDef Range: 0 .. 100 Default value: 0 Post-Build Variant Value: true Value Configuration Class: Pre-compile time: X↩ : VARIANT-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-P↩ OST-BUILD Scope / Dependency: scope: local: dependency: This parameter exists only if atleast one SPI Sequence is referenced in CanTrcv↩ SpiSequence.	This configuration is done indirectly in Ae Driver
ECUC_CanTrcv_00145	N/S	Container Name: CanTrcvDio↩ Access Parent Container: Can↩ TrcvAccess Description: Container gives CAN transceiver driver information about accessing ports and port pins. In addition relation between CAN transceiver hardware pin names and Dio port access information is given. If a CAN transceiver hardware has no Dio interface, there is no instance of this container. Configuration Parameters	Only apply for Hardware support access via DIO
ECUC_CanTrcv_00157	N/S	Container Name: CanTrcvDio↩ ChannelAccess Parent Container↩ : CanTrcvDioAccess Description↩ : Container gives DIO channel access by single Can transceiver channel. Configuration Parameters	Only apply for Hardware support access via DIO

Requirement	Status	Description	Notes
ECUC_CanTrcv_00150	N/S	Name: CanTrcvHardware↩ InterfaceName Parent Container↩ : CanTrcvDioChannelAccess Description: CAN transceiver hardware interface name. It is typically the name of a pin. From a Dio point of view it is either a port, a single channel or a channel group. Depending on this fact either CANTRCV_DIO↩ _PORT_SYMBOLIC_NAME or CANTRCV_DIO_CHAN↩ NEL_SYMBOLIC_NAME or CANTRCV_DIO_CHANNEL↩ _GROUP_SYMBOLIC_NAME shall reference a Dio configuration. The CAN transceiver driver implementation description shall list up this name for the appropriate CAN transceiver hardware. Multiplicity: 1 Type: Ecuc↩ StringParamDef Default value: – maxLength: – minLength: – regularExpression: – Post-Build Variant Value: false Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local	Only apply for Hardware support access via DIO
ECUC_CanTrcv_00149	N/S	Name: CanTrcvDioSymNameRef Parent Container: CanTrcv↩ DioChannelAccess Description: Choice Reference to a DIO Port, DIO Channel or DIO Channel Group. This reference replaces the CANTRCV_DIO_PORT_SY↩ M_NAME, CANTRCV_DIO↩ _CHANNEL_SYM_NAME and CANTRCV_DIO_GROUP_SY↩ M_NAME references in the Can Trcv SWS. Multiplicity: 1 Type: Choice reference to [DioChannel , DioChannelGroup , DioPort] Post-Build Variant Value: false Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency	Only apply for Hardware support access via DIO

Requirement	Status	Description	Notes
ECUC_CanTrcv_00190	N/S	Name: CanTrcvTimerType Parent Container: CanTrcv↩ General Description: Type of the Time Service Predefined Timer. Multiplicity: 0..1 Type: Ecuc↩ EnumerationParamDef Range: None: None Timer_1us16bit: 16 bit 1us timer Post-Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local	Applicable only for platforms which need timeout monitors
ECUC_CanTrcv_00191	N/S	Name: CanTrcvWaitTime Parent Container: CanTrcvGeneral Description: Wait time for transceiver state changes in seconds. Multiplicity: 0..1 Type: EcucFloatParamDef Range: [0 .. 2.55E-4] Default value: – Post-↩ Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local	Applicable only for platforms which need timeout monitors
ECUC_CanTrcv_00169	N/S	Name: CanTrcvBaudRate Parent Container: CanTrcvPartialNetwork Description: Indicates the data transfer rate in kbps. Multiplicity↩ : 1 Type: EcucIntegerParam↩ Def Range: 0 .. 12000 Default value: – Post-Build Variant Value↩ : true Value Configuration Class↩ : Pre-compile time: X: VARIAN↩ T-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-POST-BUILD Scope / Dependency: scope: local↩ : dependency: Although WUF with DLC=0 is technically possible, it is explicitly not wanted.	Applicable only for platforms which support Selective wakeup feature

Requirement	Status	Description	Notes
ECUC_CanTrcv_00164	N/S	Name: CanTrcvPnCanIdIs↩ Extended Parent Container: Can↩ TrcvPartialNetwork Description: Indicates whether extended or standard ID is used. TRUE = Extended Can identifier is used. FALSE = Standard Can identifier is used Multiplicity: 1 Type: Ecuc↩ BooleanParamDef Default value: false Post-Build Variant Value: true Value Configuration Class: Pre-compile time: X: VARIAN↩ T-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-POST-BUILD Scope / Dependency: scope: local	Only apply for Hardware support Selective Wakeup
ECUC_CanTrcv_00172	N/S	Name: CanTrcvPnEnabled Parent Container: CanTrcvPartialNetwork Description: Indicates whether the selective wake-up function is enabled or disabled in HW.: TR↩ UE = Selective wakeup feature is enabled in the transceiver hardware FALSE = Selective wakeup feature is disabled in the transceiver hardware Multiplicity: 1 Type: Ecuc↩ BooleanParamDef Default value↩ : false Post-Build Variant Value↩ : true Value Configuration Class↩ : Pre-compile time: X: VARIAN↩ T-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-POST-BUILD Scope / Dependency: scope: local	Only apply for Hardware support Selective Wakeup
ECUC_CanTrcv_00163	N/S	Name: CanTrcvPnFrameCanId Parent Container: CanTrcv↩ PartialNetwork Description: CAN ID of the Wake-up Frame (WUF). Multiplicity: 1 Type: EcucInteger↩ ParamDef Range: 0 .. 4294967295 Default value: – Post-Build Variant Value: true Value Configuration Class: Pre-compile time: X: V↩ ARIANT-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-P↩ OST-BUILD Scope / Dependency: scope: local	Only apply for Hardware support Selective Wakeup

Requirement	Status	Description	Notes
ECUC_CanTrcv_00162	N/S	Name: CanTrcvPnFrameCanId↩ Mask Parent Container: Can↩ TrcvPartialNetwork Description: ID Mask for the selective activation of the transceiver. It is used to enableFrame Wake-up (WUF) on a group of IDs. Multiplicity: 1 Type↩ : EcucIntegerParamDef Range: 0 .. 4294967295 Default value: – Post-Build Variant Value: true Value Configuration Class: Pre-compile time: X: VARIANT-PRE-COM↩ PILE Link time: X: VARIANT-↩ LINK-TIME Post-build time: X↩ : VARIANT-POST-BUILD Scope / Dependency: scope: local	Only apply for Hardware support Selective Wakeup
ECUC_CanTrcv_00168	N/S	Name: CanTrcvPnFrameDlc Parent Container: CanTrcvPartial↩ Network Description: Data Length of the Wake-up Frame (WUF). Multiplicity: 1 Type: EcucInteger↩ ParamDef Range: 0 .. 8 Default value: – Post-Build Variant Value↩ : true Value Configuration Class↩ : Pre-compile time: X: VARIAN↩ T-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-POST-BUILD Scope / Dependency: scope: local	Only apply for Hardware support Selective Wakeup
ECUC_CanTrcv_00170	N/S	Name: CanTrcvPowerOnFlag Parent Container: CanTrcvPartial↩ Network Description: Description↩ : Indicates if the Power On Reset (POR) flag is available and is managed by the transceiver.: TR↩ UE = Supported by Hardware. F↩ ALSE = Not supported by Hardware Multiplicity: 1 Type: Ecuc↩ BooleanParamDef Default value↩ : false Post-Build Variant Value↩ : true Value Configuration Class↩ : Pre-compile time: X: VARIAN↩ T-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-POST-BUILD Scope / Dependency: scope: local	Only apply for Hardware support Selective Wakeup

Requirement	Status	Description	Notes
ECUC_CanTrcv_00165	N/S	Container Name: CanTrcv↩ PnFrameDataMaskSpec Parent Container: CanTrcvPartialNetwork Description: Defines data payload mask to be used on the received payload in order to determine if the transceiver must be woken up by the received Wake-up Frame (WUF). Configuration Parameters	Applicable only for platforms which support Selective wakeup feature
ECUC_CanTrcv_00166	N/S	Name: CanTrcvPnFrameDataMask Parent Container: CanTrcvPn↩ FrameDataMaskSpec Description↩ : Defines the n byte (Byte0 = LSB) of the data payload mask to be used on the received payload in order to determine if the transceiver must be woken up by the received Wake-up Frame (W↩ UF). Multiplicity: 1 Type: Ecuc↩ IntegerParamDef Range: 0 .. 255 Default value: – Post-Build Variant Value: true Value Configuration Class: Pre-compile time: X↩ : VARIANT-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-P↩ OST-BUILD Scope / Dependency: scope: local	Applicable only for platforms which support Selective wakeup feature
ECUC_CanTrcv_00167	N/S	Name: CanTrcvPnFrameData↩ MaskIndex Parent Container: CanTrcvPnFrameDataMaskSpec Description: holds the position n in frame of the mask-part Multiplicity: 1 Type: EcucInteger↩ ParamDef Range: 0 .. 7 Default value: – Post-Build Variant Value: true Value Configuration Class: Pre-compile time: X: VARIAN↩ T-PRE-COMPILE Link time: X: VARIANT-LINK-TIME Post-build time: X: VARIANT-POST-BUILD Scope / Dependency: scope: local	Applicable only for platforms which support Selective wakeup feature
ECUC_CanTrcv_00183	N/S	Container Name: CanTrcvSpi↩ Access Parent Container: Can↩ TrcvAccess Description: Container gives CAN transceiver driver information about accessing Spi. If a C↩ AN transceiver hardware has no Spi interface, there is no instance of this container. Configuration Parameters	This configuration is done indirectly in Ae Driver

Requirement	Status	Description	Notes
ECUC_CanTrcv_00144	N/S	Container Name: CanTrcvSpi↩ Sequence Parent Container: Can↩ TrcvSpiAccess Description: Container gives CAN transceiver driver information about one SPI sequence. One SPI sequence used by CAN transceiver driver is in exclusive use for it. No other driver is allowed to access this sequence. CAN transceiver driver may use one sequence to access n CAN transceiver hardwares chips of the same type or n sequences are used to access one single CAN transceiver hardware chip. If a CAN transceiver hardware has no SPI interface, there is no instance of this container. Configuration Parameters	This configuration is done indirectly in Ae Driver
ECUC_CanTrcv_00176	N/S	Name: CanTrcvSpiAccess↩ Synchronous Parent Container: CanTrcvSpiSequence Description: This parameter is used to define whether the access to the Spi sequence is synchronous or asynchronous.: true: SPI access is synchronous. false: SPI access is asynchronous. Multiplicity: 0..1 Type: EcucBooleanParamDef Default value: false Post-Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local	This configuration is done indirectly in Ae Driver

Requirement	Status	Description	Notes
ECUC_CanTrcv_00151	N/S	Name: CanTrcvSpiSequenceName Parent Container: CanTrcvSpi↩ Sequence Description: Reference to a Spi sequence configuration container. Multiplicity: 0..* Type↩ : Symbolic name reference to [SpiSequence] Post-Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time↩ : X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X↩ : All Variants Link time: – Post-build time: – Scope / Dependency↩ : scope: local: dependency: Spi↩ Sequence	This configuration is done indirectly in Ae Driver

3.5 Driver Limitations

None

3.6 Driver usage and configuration tips

3.6.1 How to configure the SPI sequence via AE module

Ae module is used to access CAN transceiver hardware connected via SPI. This SPI sequence shalln't configure in CanTrcv module, It have to configure in Ae module.

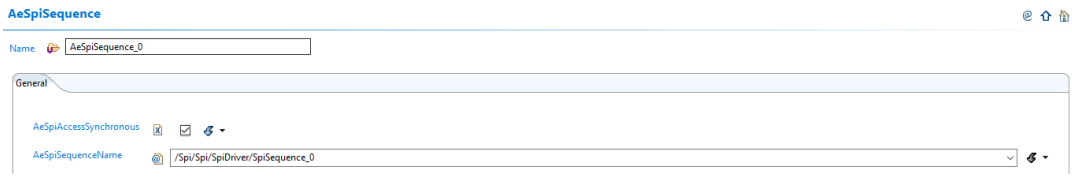

Figure 3.1 Reference to a Spi sequence configuration in Ae module over EB tresos.

SPI Configuration:
Configuration SPI sequence:

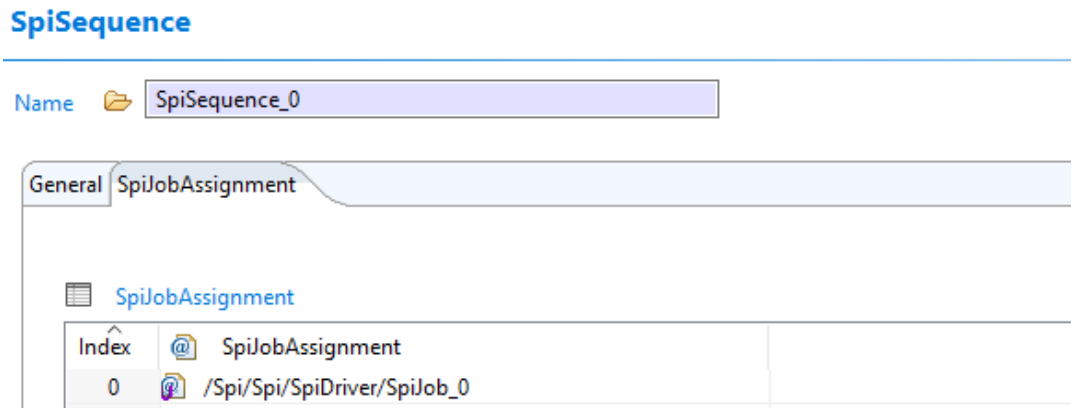

Figure 3.2 Configuration SPI sequence.

Configuration SPI Job:
In Spi Job tab, Need to configure the external device and asignment to Spi channel.

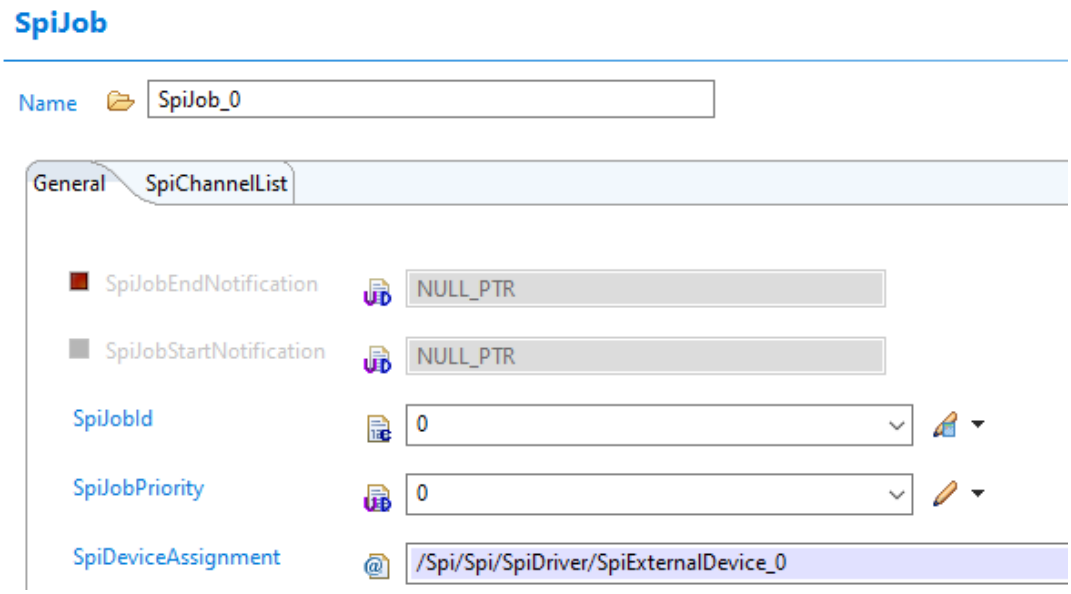

Figure 3.3 Configuration SPI job.

In Spi External Device tab:

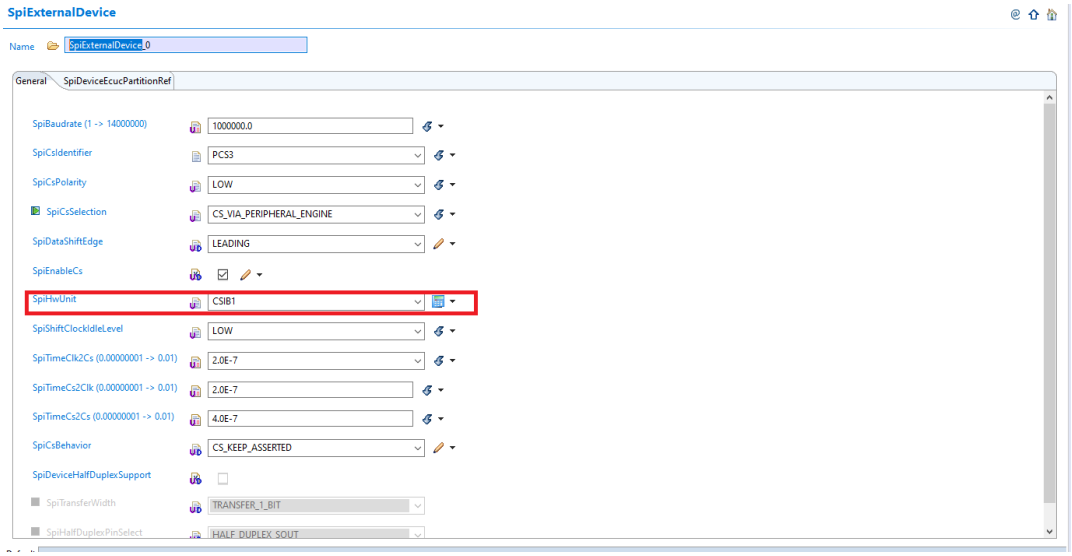

Figure 3.4 Configuration SPI External Device.

Note: SPI baudrate supported range is [4 MHz-10 MHz].
Need to make sure that the SpiHwUnit must refer to the Spi Phy Uint tab and the SpiPhyUnitMapping is LPSPI_1

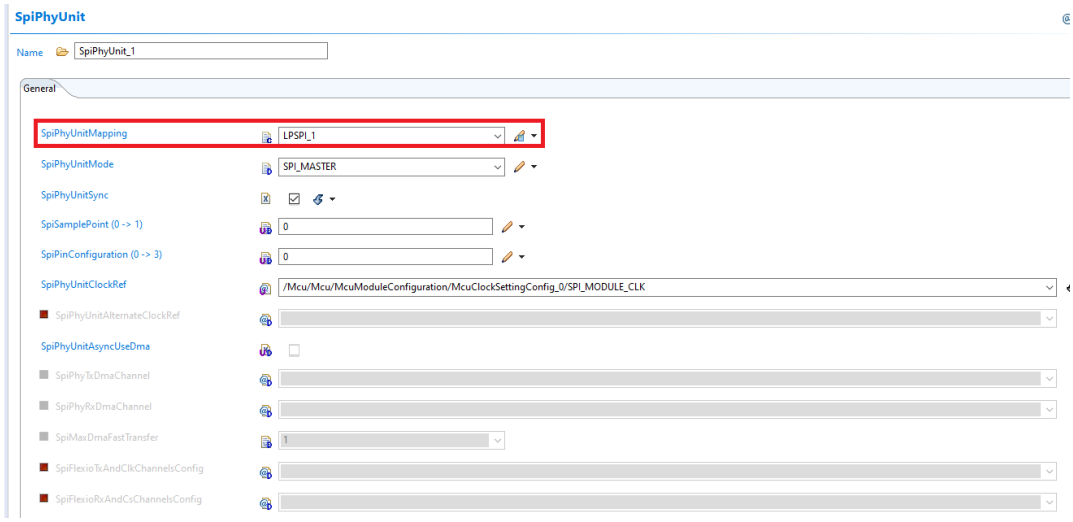

Figure 3.5 Configuration SPI channel.

In SpiChannelList tab:
Need to assignment to the Spi Channel and configure for SpiChannel as below:

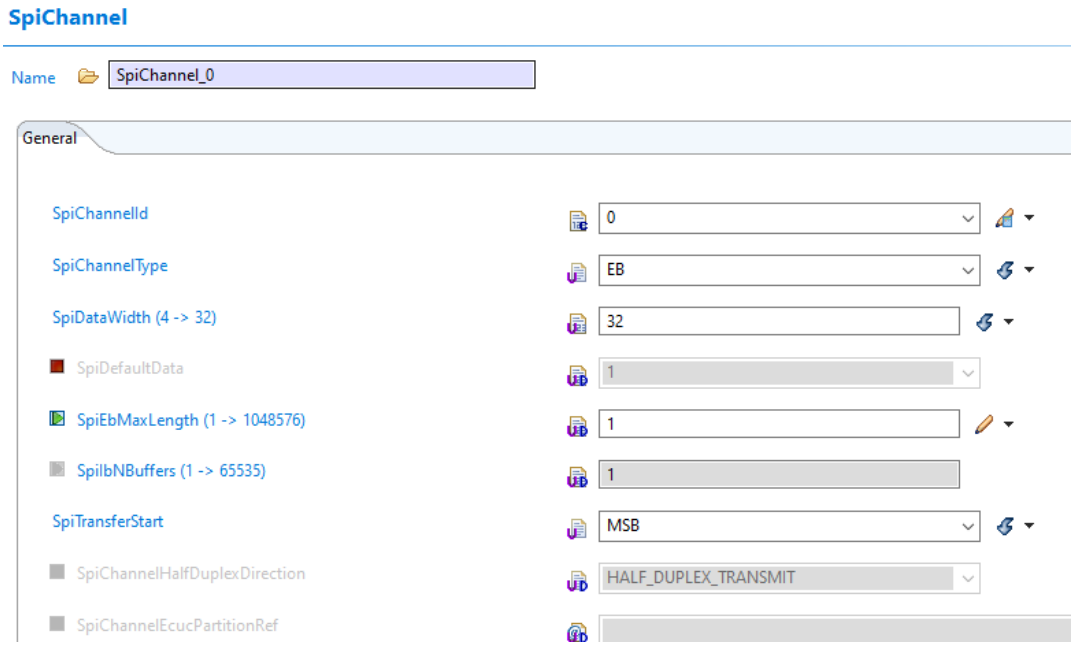

Figure 3.6 Configuration SPI channel.

Port Configuration:
Need to configure Pins for SPI in order communicates with CAN PHY interface.

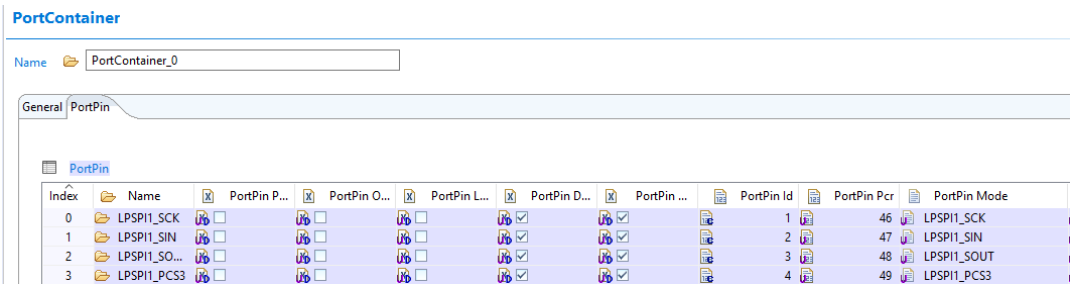

Figure 3.7 Configuration SPI Pins.

MCU Configuration:
Make sure that LPSPI_1 must enable clock.

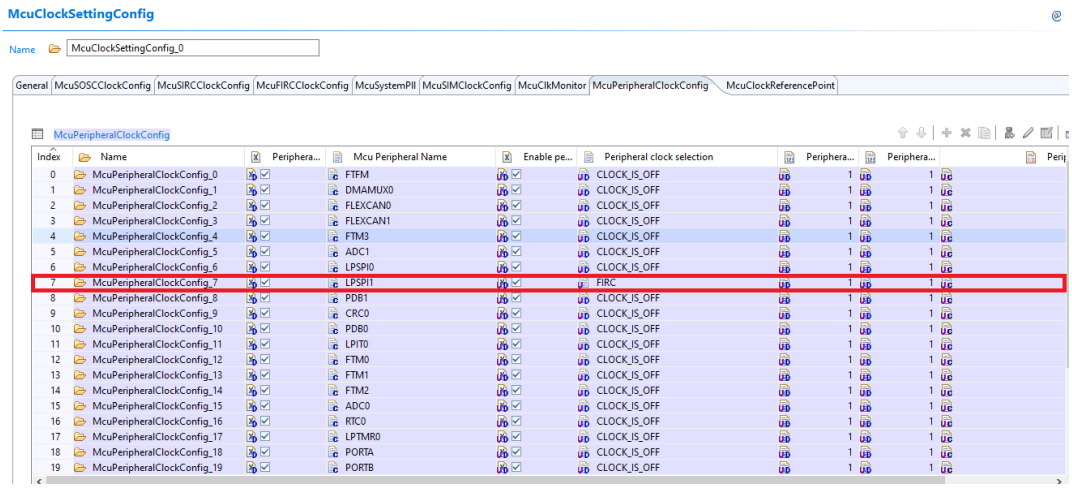

Figure 3.8 Enalbe the clock for LPSPI_1.

3.6.2 Enable CANPHY at MCU driver

- VDDC power supply and low voltate detect interrupt on VDDC must be enabled:

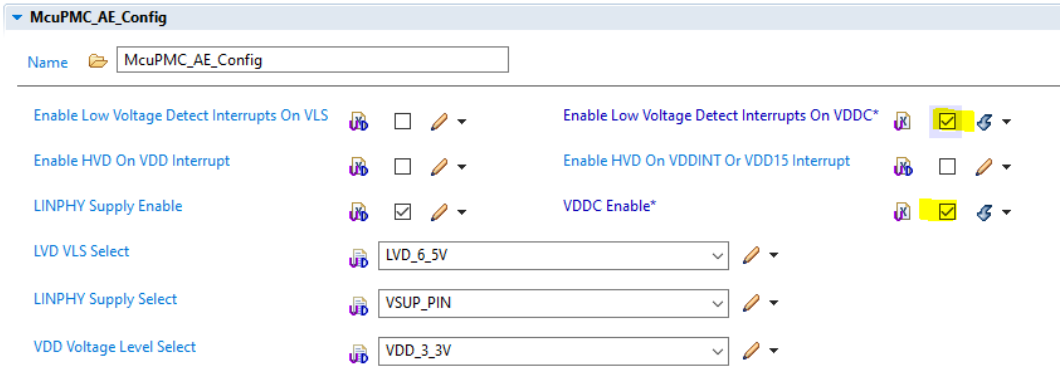

Figure 3.9 Enalbe VDDC power supply and low voltage detection.

- Enable CANPHY:

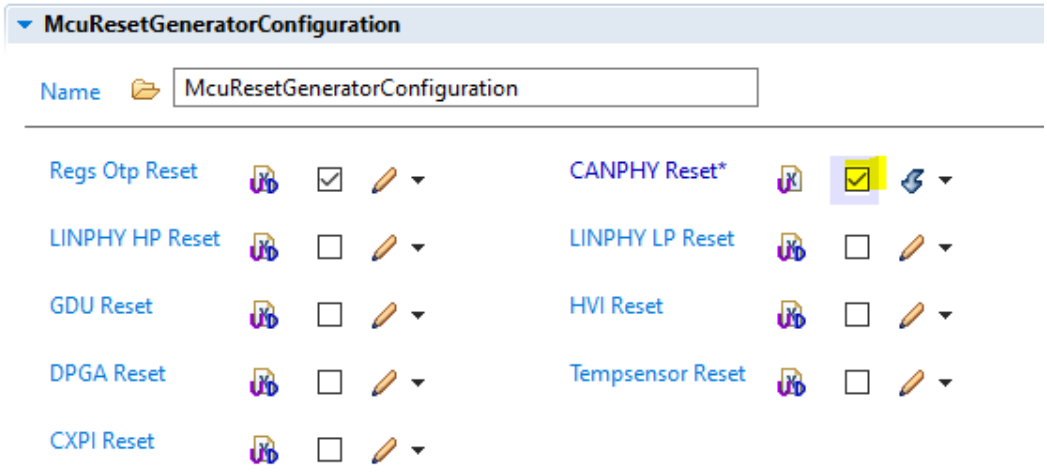

Figure 3.10 Enable CANPHY.

3.6.3 Wakeup notification handling

- The CanTrcv Driver handles only event detected by CANPHY which is CAN Wakeup Pattern.

- In order to use CanTrcv wakeup feature, the steps below can be followed:
  Step 1. Enable wakeup feature on CanTrcv driver:

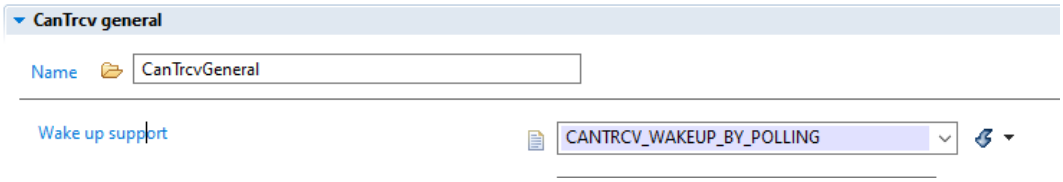

Figure 3.11 Enable Wakeup Feature.

Figure 3.12 Enable Wakeup for individual Transceiver channel.

Step 2. Configure Icu channel for MCU which will capture signals on "INTERRUPT_OUT" pin sent from AEC. The "INTERRUPT_OUT" pin is connected fixly to PTD3 pin of MCU inside chip-package. The notifications can be events/faults.
The Icu channel should be configured for PTD3 pin.

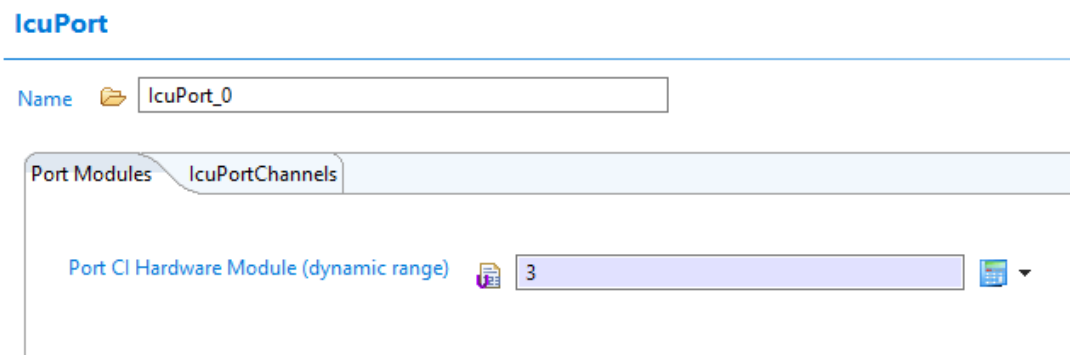

Figure 3.13 IcuPort channel configuration.

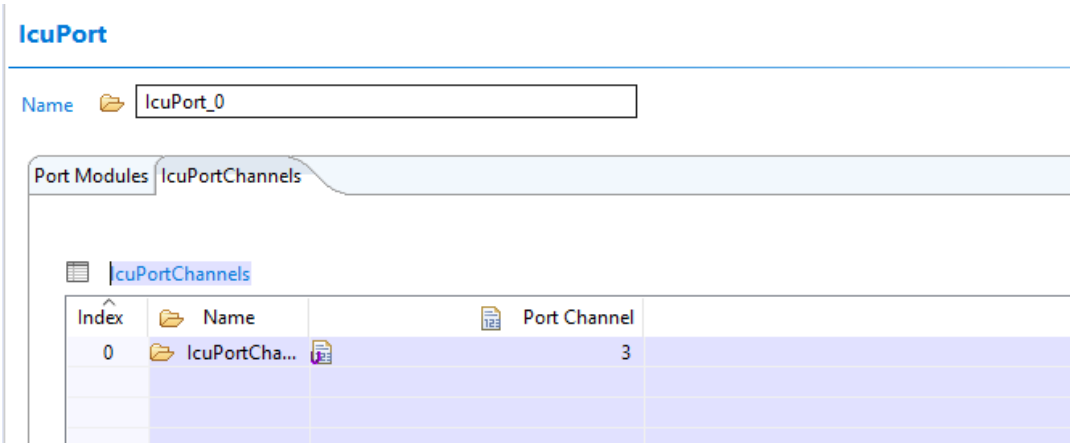

Figure 3.14 IcuPort channel configuration.

IcuSignalNotification must be actived and referenced to "AEC_Common_IRQHandler" declared in AE driver.

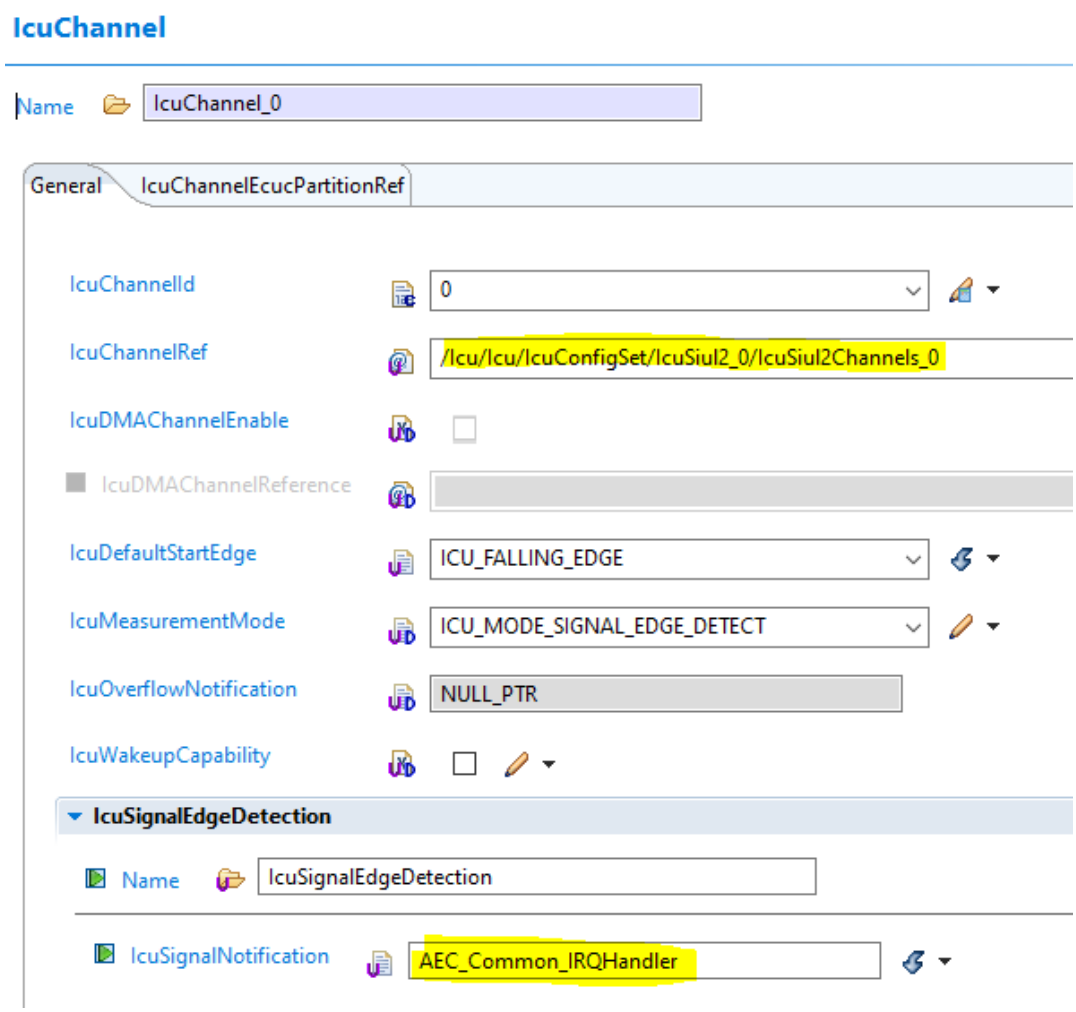

Figure 3.15 Icu channel configuration.

Note: the PTD3 pin must be also configured in PORT driver

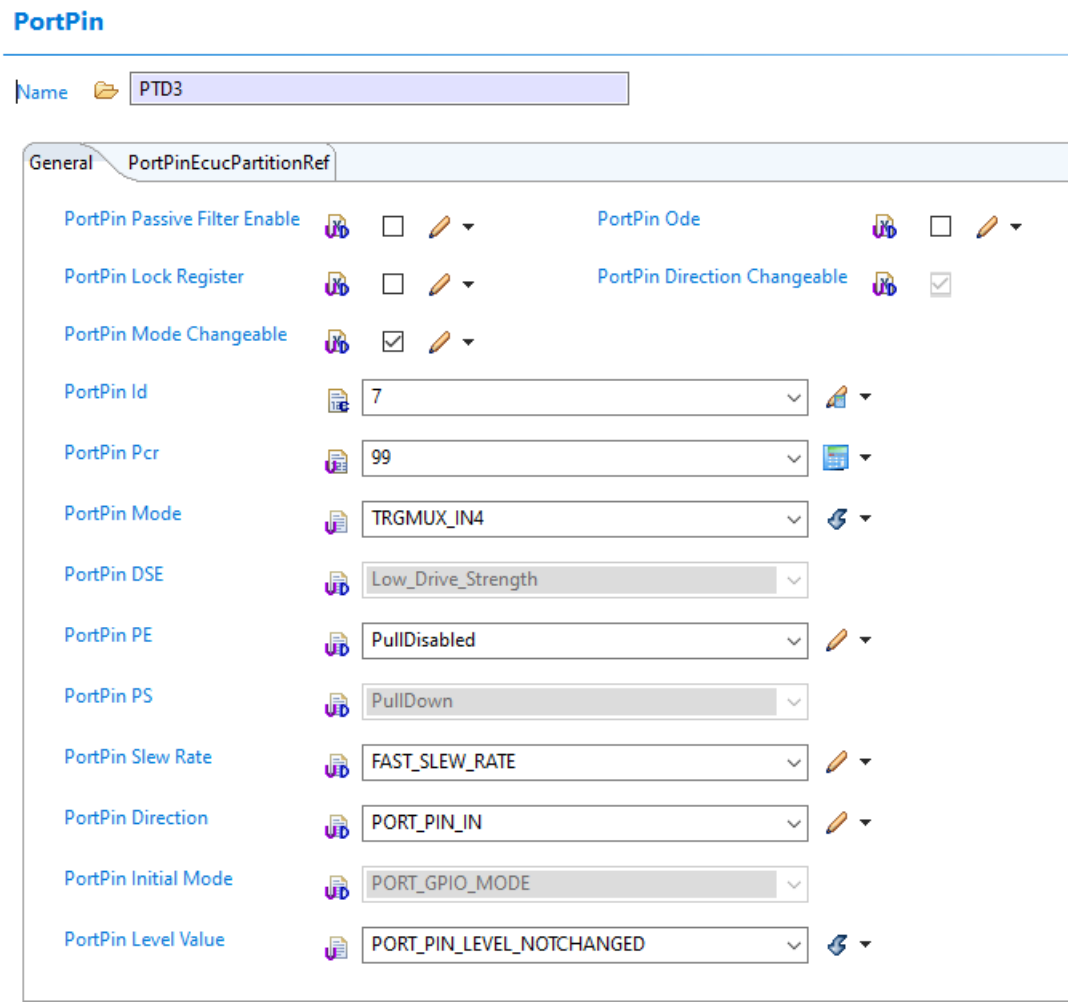

Figure 3.16 PTD3 configuration.

So now, any events/faults(enabled notification to AEC) will cause an ICU channel interrupt and we need to decide if a Icu's notification(AEC_Common_IRQHandler) called in next step.
Step 3. Configures CanTrcvIcuChannelRef to reference to corresponding Icu channel configured at above step. so the enabling/disabling the Icu notification will be done in CanTrcv Driver. If CanTrcvIcuChannelRef is not configured the Icu notification can be handled by the user in application code.

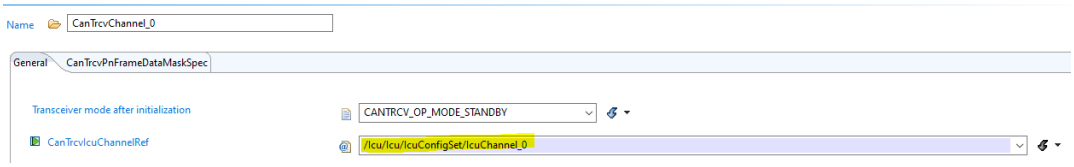

Figure 3.17 CanTrcvIcuChannelRef configuration.

step 4. Add user's notification callback which will be called after a wakeup notification sent from AEC.

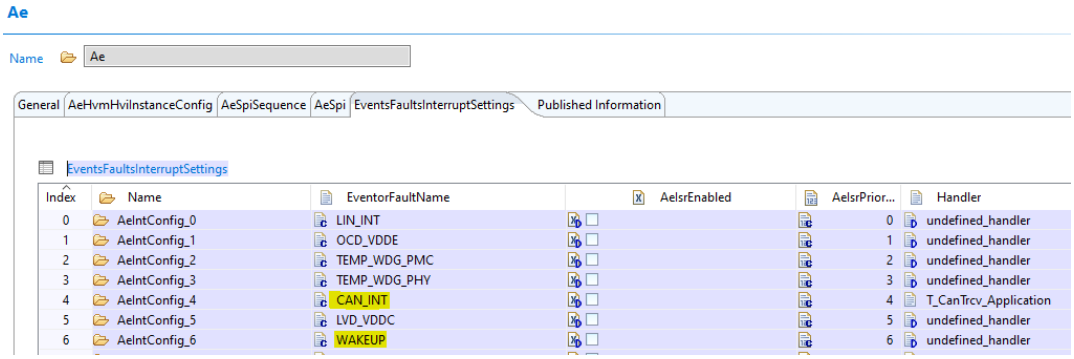

Figure 3.18 AE notification.

"CAN_INT" should be used if the user want to be notified only when a Wakeup Pattern detected by CANPHY, in this callback function, the user should call CanTrcv_43_AE_CheckWakeup to set a Wakeup event to EcuM module.
"LVD_VDDC" should be used by user which is notified if a low voltage detected on VDDC supply for CANPHY. In order to send a wakeup event to EcuM module, CanTrcvWakeupSourceRef should be activated and referenced to EcuM module:

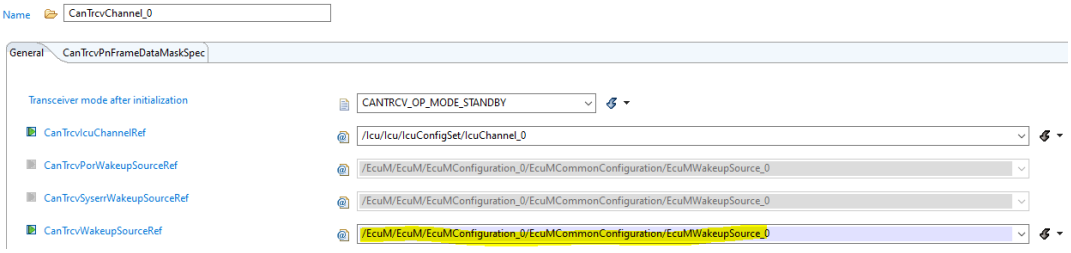

Figure 3.19 CanTrcvWakeupSourceRef configuration.

3.7 Runtime errors

The CanTrcv driver generates the following errors at runtime:

3.7.1 Runtime Errors

Function	Error code	Condition triggering the error
CanTrcv_Init	CANTRCV_E_NO_TRCV_C↩ ONTROL	No/incorrect communication to transceiver.
CanTrcv_SetOpMode	CANTRCV_E_NO_TRCV_C↩ ONTROL	No/incorrect communication to transceiver.
CanTrcv_GetOpMode	CANTRCV_E_NO_TRCV_C↩ ONTROL	No/incorrect communication to transceiver.
CanTrcv_GetBusWuReason	CANTRCV_E_NO_TRCV_C↩ ONTROL	No/incorrect communication to transceiver.

Function	Error code	Condition triggering the error
CanTrcv_ SetWakeupMode	CANTRCV_E_NO_TRCV_C↩ ONTROL	No/incorrect communication to transceiver.
CanTrcv_DeInit	CANTRCV_E_NO_TRCV_C↩ ONTROL	No/incorrect communication to transceiver.
CanTrcv_MainFunction↩ Diagnostics	CANTRCV_E_BUS_ERROR	A CAN bus error occured during communication.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: *#define <Mip>Conf_<Container_ShortName>_<Container_ID>*

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

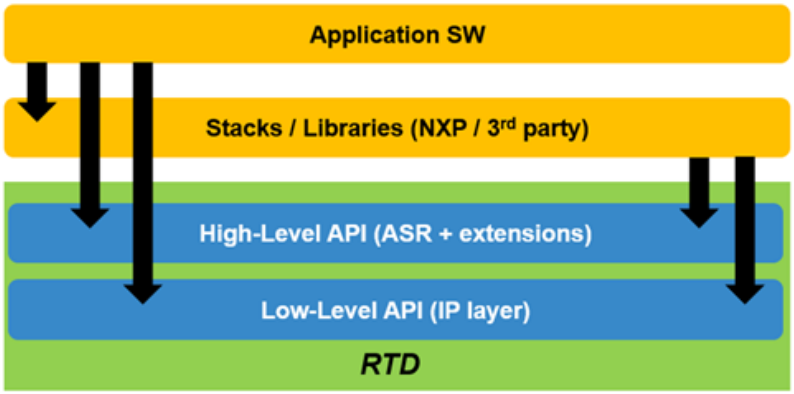

Figure 3.20 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
  1. Initialization of the device clock tree so other periphery can be initialized
  2. Configuration of the pinout so peripherals with external connections can communicate with the outside world

3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
4. Initialization of specific drivers
5. Runtime functionality
6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

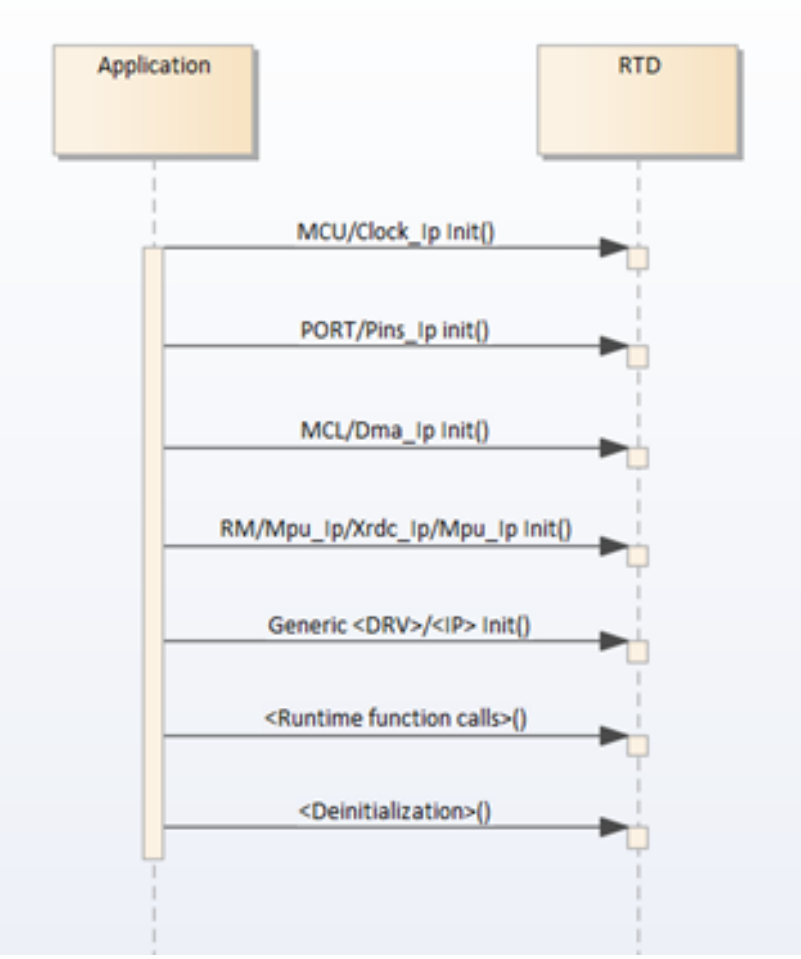

Figure 3.21 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

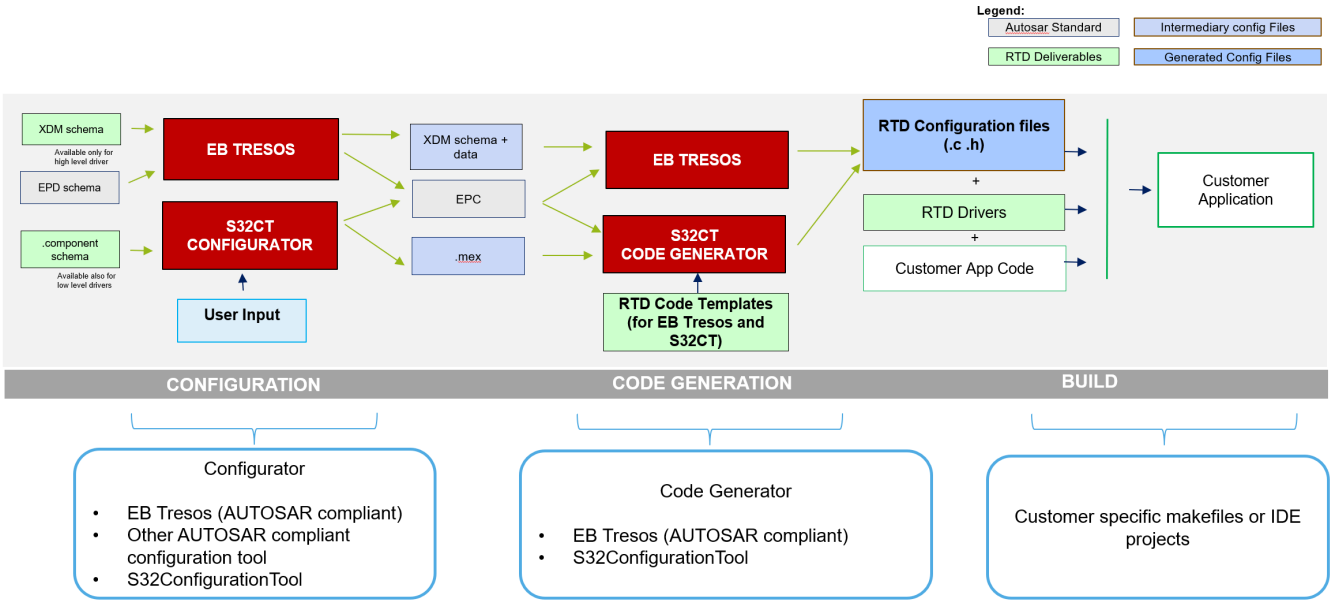

Figure 4.1 Configuration generation flow

- CanTrcv

4.1 CanTrcv

- Module CanTrcv

- Container CanTrcvGeneral
  - Parameter CanTrcvWakeUpSupport
  - Parameter CanTrcvMainFunctionDiagnosticsPeriod
  - Parameter CanTrcvMainFunctionPeriod
  - Parameter CanTrcvDevErrorDetect
  - Parameter CanTrcvDisableDiagnosticEventManager
  - Parameter CanTrcvMultiPartitionSupport
  - Parameter CanTrcvTimerType
  - Parameter CanTrcvWaitTime
  - Parameter CanTrcvVersionInfoApi
  - Parameter CanTrcvIndex
  - Reference CanTrcvEcucPartitionRef
- Container CanTrcvConfigSet
  - Parameter CanTrcvSPICommRetries
  - Parameter CanTrcvSPICommTimeout
  - Container CanTrcvChannel
    - Parameter CanTrcvInitState
    - Parameter CanTrcvChannelId
    - Parameter CanTrcvAbstractCanIfId
    - Parameter CanTrcvMaxBaudrate
    - Parameter CanTrcvChannelUsed
    - Parameter CanTrcvControlsPowerSupply
    - Parameter CanTrcvWakeupByBusUsed
    - Parameter CanTrcvHwPnSupport
    - Reference CanTrcvIcuChannelRef
    - Reference CanTrcvPorWakeupSourceRef
    - Reference CanTrcvSyserrWakeupSourceRef
    - Reference CanTrcvWakeupSourceRef
    - Reference CanTrcvChannelEcucPartitionRef
    - Container CanTrcvAccess
    - Container CanTrcvDemEventParameterRefs
      - Reference CANTRCV_E_BUS_ERROR
    - Container CanTrcvPartialNetwork
      - Parameter CanTrcvBaudRate
      - Parameter CanTrcvPnFrameCanId
      - Parameter CanTrcvPnFrameCanIdMask

- Parameter CanTrcvPnFrameDlc
- Parameter CanTrcvPnCanIdIsExtended
- Parameter CanTrcvPnEnabled
- Parameter CanTrcvBusErrFlag
- Parameter CanTrcvPowerOnFlag
- Container CanTrcvPnFrameDataMaskSpec
  - Parameter CanTrcvPnFrameDataMask
  - Parameter CanTrcvPnFrameDataMaskIndex

– Container CommonPublishedInformation

* Parameter ArReleaseMajorVersion
* Parameter ArReleaseMinorVersion
* Parameter ArReleaseRevisionVersion
* Parameter ModuleId
* Parameter SwMajorVersion
* Parameter SwMinorVersion
* Parameter SwPatchVersion
* Parameter VendorApiInfix
* Parameter VendorId

4.1.1 Module CanTrcv

This container holds the configuration of a single CanTrcv Driver.

Included containers:

- CanTrcvGeneral
- CanTrcvConfigSet
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-LINK-TIME, VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container CanTrcvGeneral

This container holds parameters related to each CanTrcv Driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter CanTrcvWakeUpSupport

CANTRCV_WAKEUP_BY_POLLING: Wake up by polling.

CANTRCV_WAKEUP_NOT_SUPPORTED: Wake up not supported.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CANTRCV_WAKEUP_NOT_SUPPORTED
literals	['CANTRCV_WAKEUP_BY_POLLING', 'CANTRCV_WAKEUP_NOT_↩ SUPPORTED']

4.1.4 Parameter CanTrcvMainFunctionDiagnosticsPeriod

This parameter describes the period for cyclic call to CanTrcv_MainFunctionDiagnostics.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.0
max	255.0
min	0.0

4.1.5 Parameter CanTrcvMainFunctionPeriod

This parameter describes the period for cyclic call to CanTrcv_MainFunction.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.0
max	255.0
min	0.0

4.1.6 Parameter CanTrcvDevErrorDetect

Switches the Development Error Detection and Notification: ON or OFF.

When this option is OFF code size is reduced, but no error detection is available.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.7 Parameter CanTrcvDisableDiagnosticEventManager

Enable or Disable the API for reporting the Dem Error.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.8 Parameter CanTrcvMultiPartitionSupport

Enable Maps CanTrcv driver to multiple EcuC partitions to make the modules API available in this partition. The CanTrcv driver will operate as an independent instance in each of the partitions.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.9 Parameter CanTrcvTimerType

Type of the Time Service Predefined Timer.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	None
literals	['None', 'Timer_1us16bit']

4.1.10 Parameter CanTrcvWaitTime

Wait time for transceiver state changes in seconds.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.0
max	2.55E-4
min	0.0

4.1.11 Parameter CanTrcvVersionInfoApi

Switches the CanTrcv_GetVersionInfo() API: ON or OFF.

When this option is ON, the driver supports API for getting Version information of the Driver.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.12 Parameter CanTrcvIndex

Specifies the InstanceId of this module instance.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.13 Reference CanTrcvEcucPartitionRef

Maps the CAN transceiver driver to zero or multiple ECUC partitions to make the modules API available in this partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.14 Container CanTrcvConfigSet

This is the multiple configuration set container for CanTrcv Driver.

Included subcontainers:

- CanTrcvChannel

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.15 Parameter CanTrcvSPICommRetries

Indicates the maximum number of communication retries in case of a failed SPI communication (applies both to timed out communication and to errors/NACK in the response data).

If configured value is '0', no retry is allowed (communication is expected to succeed at first try).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.16 Parameter CanTrcvSPICommTimeout

Indicates the maximum time allowed to the CanTrcv to reply (either positively or negatively) to a SPI command.

Timeout is configured in milliseconds. Timeout value of '0' means no specific timeout is to be used by CanTrcv and the communication is executed at the best of the SPI HW capacity.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	100
min	0

4.1.17 Container CanTrcvChannel

Defines a structure to initialize the selected CAN transceiver (channel).

Included subcontainers:

- CanTrcvAccess
- CanTrcvDemEventParameterRefs
- CanTrcvPartialNetwork

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.18 Parameter CanTrcvInitState

State of CAN transceiver after call to CanTrcv_Init.

- Normal mode after Init

- Sleep mode after Init

- Standby mode after Init (Default)

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	CANTRCV_OP_MODE_STANDBY
literals	['CANTRCV_OP_MODE_SLEEP', 'CANTRCV_OP_MODE_STANDBY']

4.1.19 Parameter CanTrcvChannelId

Unique identifier of the CAN Transceiver Channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	0

4.1.20 Parameter CanTrcvAbstractCanIfId

Vendor specific:The abstract CanIf TransceiverId used when Driver invokes the callback functions from CanIf.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	0

4.1.21 Parameter CanTrcvMaxBaudrate

Indicates the data transfer rate in kbps. Maximum data transfer rate in kbps for transceiver hardware type.

For CAN the maximum data transfer rate is 500 kbps, for CAN FD 2000 kbps.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	5000
max	12000
min	0

4.1.22 Parameter CanTrcvChannelUsed

Shall the related CAN transceiver be used?

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.23 Parameter CanTrcvControlsPowerSupply

Is ECU power supply controlled by this transceiver?

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.24 Parameter CanTrcvWakeupByBusUsed

Is wake up by bus supported?

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.25 Parameter CanTrcvHwPnSupport

Indicates whether the HW supports the selective wake-up functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.26 Reference CanTrcvIcuChannelRef

Reference to the IcuChannel to enable/disable the interrupts for wakeups.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Icu/IcuConfigSet/IcuChannel

4.1.27 Reference CanTrcvPorWakeupSourceRef

Symbolic name reference to specify the wakeup sources that should be used in the calls to EcuM_SetWakeupEvent.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuM/EcuMConfiguration/EcuMCommon↩ Configuration/EcuMWakeupSource

4.1.28 Reference CanTrcvSyserrWakeupSourceRef

Symbolic name reference to specify the wakeup sources that should be used in the calls to EcuM_SetWakeupEvent.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuM/EcuMConfiguration/EcuMCommon↩ Configuration/EcuMWakeupSource

4.1.29 Reference CanTrcvWakeupSourceRef

Reference to a wakeup source in the EcuM configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False

Property	Value
destination	/AUTOSAR/EcucDefs/EcuM/EcuMConfiguration/EcuMCommon↩ Configuration/EcuMWakeupSource

4.1.30 Reference CanTrcvChannelEcucPartitionRef

Maps the CAN transceiver configuration to zero or one ECUC partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.31 Container CanTrcvAccess

Gives CanTrcv Driver information about access to a single CAN transceiver.

Included choices:

- CanTrcvSpiAccess
- CanTrcvDioAccess

Property	Value
type	ECUC-CHOICE-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.32 Container CanTrcvDemEventParameterRefs

Container for the references to DemEventParameter elements which shallbe invoked using the API Dem_SetEventStatus in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId symbolic value. The standardized errors are provided in this container and can be extended by vendor-specific error references.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.33 Reference CANTRCV_E_BUS_ERROR

Reference to the DemEventParameter which shall be issued when bus error has occurred.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Dem/DemConfigSet/DemEventParameter

4.1.34 Container CanTrcvPartialNetwork

Container gives CAN transceiver driver information about the configuration of Partial Networking functionality.

Included subcontainers:

- CanTrcvPnFrameDataMaskSpec

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.35 Parameter CanTrcvBaudRate

Indicates the data transfer rate in kbps.

Supported values:

- 50 kBits

- 100 kBits

- 125 kBits

- 250 kBits

- 500 kBits

- 1000 kBits

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	50
max	12000
min	0

4.1.36 Parameter CanTrcvPnFrameCanId

CAN ID of the Wake-up Frame (WUF).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.37 Parameter CanTrcvPnFrameCanIdMask

ID Mask for the selective activation of the transceiver. It is used to enable-Frame Wake-up (WUF) on a group of IDs.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.38 Parameter CanTrcvPnFrameDlc

Data Length of the Wake-up Frame (WUF).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	8
min	0

4.1.39 Parameter CanTrcvPnCanIdIsExtended

Indicates whether extended or standard ID is used.

TRUE = Extended Can identifier is used.

FALSE = Standard Can identifier is used

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.40 Parameter CanTrcvPnEnabled

Indicates whether the selective wake-up function is enabled or disabled in HW.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.41 Parameter CanTrcvBusErrFlag

Indicates if the Bus Error (BUSERR) flag is managed by the BSW. This flag is set if a bus failure is detected by the transceiver.

TRUE = Supported by transceiver and managed by BSW. FALSE = Not managed by BSW.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.42 Parameter CanTrcvPowerOnFlag

Description: Indicates if the Power On Reset (POR) flag is available and is managed by the transceiver.

TRUE = Supported by Hardware.

FALSE = Not supported by Hardware

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.43 Container CanTrcvPnFrameDataMaskSpec

Defines data payload mask to be used on the received payload in order to determine if the transceiver must be woken up by the received wake-up frame (WUF).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	8
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-LINK-TIME: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.44 Parameter CanTrcvPnFrameDataMask

Defines the byte 0 of the data payload mask to be used on the received payload in order to determine if the transceiver must be woken up by the received Wake-up frame (WUF)..

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.45 Parameter CanTrcvPnFrameDataMaskIndex

Holds the position n in frame of the mask-part.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-LINK-TIME: LINK
	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.1.46 Container CommonPublishedInformation

Common container, aggregated by all modules.

It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.47 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.48 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7

Property	Value
max	7
min	7

4.1.49 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.50 Parameter ModuleId

Module ID of this module from Module List.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	70
max	70
min	70

4.1.51 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.52 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.53 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.54 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the Implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity >

1. It shall not be used for modules with upper multiplicity =1.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	AE

4.1.55 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-LINK-TIME: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Chapter 6

Module Documentation

6.1 Can Transceiver Driver

6.1.1 Detailed Description

Data Structures

- struct CanTrcv_43_AE_ConfigType

  Can Transceiver Configuration. *More...*
- struct CanTrcv_43_AE_TransceiverConfigType

  CanTrcv_43_AE_TransceiverConfigType. *More...*

Macros

- #define CANTRCV_43_AE_E_INVALID_TRANSCEIVER

  Development Error ID for API called with wrong parameter for the CAN transceiver.
- #define CANTRCV_43_AE_E_PARAM_POINTER

  Development Error ID for API called with null pointer parameter.
- #define CANTRCV_43_AE_E_UNINIT

  Development Error ID for API service used without initialization.
- #define CANTRCV_43_AE_E_TRCV_NOT_STANDBY

  Development Error ID for API service called in wrong transceiver operation mode (STANDBY expected)
- #define CANTRCV_43_AE_E_TRCV_NOT_NORMAL

  Development Error ID for API service called in wrong transceiver operation mode (NORMAL expected)
- #define CANTRCV_43_AE_E_PARAM_TRCV_WAKEUP_MODE

  Development Error ID for API service called with invalid parameter for TrcvWakeupMode.
- #define CANTRCV_43_AE_E_PARAM_TRCV_OPMODE

  Development Error ID for API service called with invalid parameter for OpMode.
- #define CANTRCV_43_AE_E_INIT_FAILED

  Development Error ID for Module initialization has failed, e.g. CanTrcv_Init() called with an invalid pointer in postbuild.
- #define CANTRCV_43_AE_E_NO_TRCV_CONTROL

  Runtime Error ID when No/incorrect communication to transceiver.
- #define CANTRCV_43_AE_SID_INIT

Service ID of CanTrcv_43_AE_Init.

- #define CANTRCV_43_AE_SID_SET_OPMODE

  Service ID of CanTrcv_43_AE_SetOpMode.
- #define CANTRCV_43_AE_SID_GET_OPMODE

  Service ID of CanTrcv_43_AE_GetOpMode.
- #define CANTRCV_43_AE_SID_GET_BUS_WU_REASON

  Service ID of CanTrcv_43_AE_GetBusWuReason.
- #define CANTRCV_43_AE_SID_GET_VERSION_INFO

  Service ID of CanTrcv_43_AE_GetVersionInfo.
- #define CANTRCV_43_AE_SID_SET_WAKEUP_MODE

  Service ID of CanTrcv_43_AE_SetWakeupMode.
- #define CANTRCV_43_AE_SID_CHECK_WAKEUP

  Service ID of CanTrcv_43_AE_CheckWakeup.
- #define CANTRCV_43_AE_SID_CHECK_WAKE_FLAG

  Service ID of CanTrcv_43_AE_CheckWakeFlag.
- #define CANTRCV_43_AE_SID_DEINIT

  Service ID of CanTrcv_43_AE_DeInit.
- #define CANTRCV_43_AE_SID_MAINFUNCTION

  Service ID of CanTrcv_43_AE_MainFunction.
- #define CANTRCV_43_AE_SID_MAINFUNCTION_DIAGNOSTICS

  Service ID of CanTrcv_43_AE_MainFunctionDiagnostics.

Enum Reference

- enum CanTrcv_43_AE_eDriverStatusType

  CanTrcv_43_AE_eDriverStatusType.

Function Reference

- void CanTrcv_43_AE_GetVersionInfo (Std_VersionInfoType *versioninfo)

  CAN transceiver driver get version info function. SID is 0x04.
- void CanTrcv_43_AE_Init (const CanTrcv_43_AE_ConfigType *ConfigPtr)

  Initializes CanTrcv module. SID 0x00.
- Std_ReturnType CanTrcv_43_AE_SetOpMode (uint8 Transceiver, CanTrcv_TrcvModeType OpMode)

  Sets the mode of the Transceiver to the value OpMode. SID 0x01.
- Std_ReturnType CanTrcv_43_AE_GetOpMode (uint8 Transceiver, CanTrcv_TrcvModeType *OpMode)

  Gets current operation mode. SID 0x02.
- Std_ReturnType CanTrcv_43_AE_GetBusWuReason (uint8 Transceiver, CanTrcv_TrcvWakeupReason↩ Type *reason)

  Gets the wakeup reason of the Transceiver and returns it in parameter Reason. SID 0x03.
- Std_ReturnType CanTrcv_43_AE_SetWakeupMode (uint8 Transceiver, CanTrcv_TrcvWakeupModeType TrcvWakeupMode)

  Enables, disables or clears wake up events of the Transceiver according to TrcvWakeupMode. SID 0x05.

- Std_ReturnType CanTrcv_43_AE_CheckWakeup (uint8 Transceiver)

  Service is called by underlying CANIF in case a wake up interrupt is detected. SID 0x07.
- Std_ReturnType CanTrcv_43_AE_CheckWakeFlag (uint8 Transceiver)

  Requests to check the status of the wakeup flag from the transceiver hardware. SID 0x0E.
- void CanTrcv_43_AE_DeInit (void)

  De-initializes the CanTrcv module. SID 0x10.

6.1.2 Data Structure Documentation

6.1.2.1 struct CanTrcv_43_AE_ConfigType

Can Transceiver Configuration.

Definition at line 211 of file CanTrcv_43_AE.h.

Data Fields

- const uint32 CanTrcv_u32PartitionId

  Configuration Partition ID.
- const CanTrcv_43_AE_TransceiverConfigType *const * CanTrcv_ppTransceivers

  Pointer to all Tranceiver channels asigned to the partition.

Field Documentation

CanTrcv_u32PartitionId

```
const uint32 CanTrcv_u32PartitionId
```

Configuration Partition ID.

Definition at line 214 of file CanTrcv_43_AE.h.

CanTrcv_ppTransceivers

```
const CanTrcv_43_AE_TransceiverConfigType* const* CanTrcv_ppTransceivers
```

Pointer to all Tranceiver channels asigned to the partition.

Definition at line 216 of file CanTrcv_43_AE.h.

6.1.2.2 struct CanTrcv_43_AE_TransceiverConfigType

CanTrcv_43_AE_TransceiverConfigType.

Definition at line 136 of file CanTrcv_43_AE_Types.h.

Data Fields

const uint8	CanTrcv_u8SwTransceiverId	The Id of Transceiver channel configured by tool configuration.
const uint8	CanTrcv_u8HwTransceiverId	The Id of Transceiver channel mapped in Hardware.

Data Fields

const uint8	CanTrcv_u8CanIfTransceiverId	The Id referred by CanIf to CanTrcv.
const boolean	CanTrcv_bIcuUsed	The Icu channel referred by the transceiver.
const uint16	CanTrcv_u16IcuChnId	The Icu channel referred by the transceiver.
const uint32	CanTrcv_WakeupSourceMask	The value report to EcuM for a wakeup event.
const uint16	CanTrcv_DemEventId	The value report to Dem for a bus error.
const boolean	CanTrcv_bWakeupByBusUsed	Specifies if wakeup is enabled on the transceiver.
const CanTrcv_TrcvModeType	CanTrcv_eInitState	The mode of Transceiver after Init.
const CanTrcv_43_AE_Ipw_TransceiverConfigType *const	CanTrcv_IpwTransceiverConfig	Pointer to Ipw configuration wrapped to specific IPV configuration.

6.1.3 Macro Definition Documentation

6.1.3.1 CANTRCV_43_AE_E_INVALID_TRANSCEIVER

```
#define CANTRCV_43_AE_E_INVALID_TRANSCEIVER
```

Development Error ID for API called with wrong parameter for the CAN transceiver.

Development errors. The Can module shall be able to detect the following errors and exceptions depending on its configuration (development/production). Development Errors shall indicate errors that are caused by erroneous usage of the Can module API. This covers API parameter checks and call sequence errors. Development error values are of type uint8.

Definition at line 140 of file CanTrcv_43_AE.h.

6.1.3.2 CANTRCV_43_AE_E_PARAM_POINTER

```
#define CANTRCV_43_AE_E_PARAM_POINTER
```

Development Error ID for API called with null pointer parameter.

Definition at line 142 of file CanTrcv_43_AE.h.

6.1.3.3 CANTRCV_43_AE_E_UNINIT

```
#define CANTRCV_43_AE_E_UNINIT
```

Development Error ID for API service used without initialization.

Definition at line 144 of file CanTrcv_43_AE.h.

6.1.3.4 CANTRCV_43_AE_E_TRCV_NOT_STANDBY

```
#define CANTRCV_43_AE_E_TRCV_NOT_STANDBY
```

Development Error ID for API service called in wrong transceiver operation mode (STANDBY expected)

Definition at line 146 of file CanTrcv_43_AE.h.

6.1.3.5 CANTRCV_43_AE_E_TRCV_NOT_NORMAL

```
#define CANTRCV_43_AE_E_TRCV_NOT_NORMAL
```

Development Error ID for API service called in wrong transceiver operation mode (NORMAL expected)

Definition at line 148 of file CanTrcv_43_AE.h.

6.1.3.6 CANTRCV_43_AE_E_PARAM_TRCV_WAKEUP_MODE

```
#define CANTRCV_43_AE_E_PARAM_TRCV_WAKEUP_MODE
```

Development Error ID for API service called with invalid parameter for TrcvWakeupMode.

Definition at line 150 of file CanTrcv_43_AE.h.

6.1.3.7 CANTRCV_43_AE_E_PARAM_TRCV_OPMODE

```
#define CANTRCV_43_AE_E_PARAM_TRCV_OPMODE
```

Development Error ID for API service called with invalid parameter for OpMode.

Definition at line 152 of file CanTrcv_43_AE.h.

6.1.3.8 CANTRCV_43_AE_E_INIT_FAILED

```
#define CANTRCV_43_AE_E_INIT_FAILED
```

Development Error ID for Module initialization has failed, e.g. CanTrcv_Init() called with an invalid pointer in postbuild.

Definition at line 158 of file CanTrcv_43_AE.h.

6.1.3.9 CANTRCV_43_AE_E_NO_TRCV_CONTROL

```
#define CANTRCV_43_AE_E_NO_TRCV_CONTROL
```

Runtime Error ID when No/incorrect communication to transceiver.

Definition at line 165 of file CanTrcv_43_AE.h.

6.1.3.10 CANTRCV_43_AE_SID_INIT

```
#define CANTRCV_43_AE_SID_INIT
```

Service ID of CanTrcv_43_AE_Init.

Service ID (APIs) for Det reporting

Definition at line 171 of file CanTrcv_43_AE.h.

6.1.3.11 CANTRCV_43_AE_SID_SET_OPMODE

```
#define CANTRCV_43_AE_SID_SET_OPMODE
```

Service ID of CanTrcv_43_AE_SetOpMode.

Definition at line 173 of file CanTrcv_43_AE.h.

6.1.3.12 CANTRCV_43_AE_SID_GET_OPMODE

```
#define CANTRCV_43_AE_SID_GET_OPMODE
```

Service ID of CanTrcv_43_AE_GetOpMode.

Definition at line 175 of file CanTrcv_43_AE.h.

6.1.3.13 CANTRCV_43_AE_SID_GET_BUS_WU_REASON

```
#define CANTRCV_43_AE_SID_GET_BUS_WU_REASON
```

Service ID of CanTrcv_43_AE_GetBusWuReason.

Definition at line 177 of file CanTrcv_43_AE.h.

6.1.3.14 CANTRCV_43_AE_SID_GET_VERSION_INFO

```
#define CANTRCV_43_AE_SID_GET_VERSION_INFO
```

Service ID of CanTrcv_43_AE_GetVersionInfo.

Definition at line 179 of file CanTrcv_43_AE.h.

6.1.3.15 CANTRCV_43_AE_SID_SET_WAKEUP_MODE

```
#define CANTRCV_43_AE_SID_SET_WAKEUP_MODE
```

Service ID of CanTrcv_43_AE_SetWakeupMode.

Definition at line 181 of file CanTrcv_43_AE.h.

6.1.3.16 CANTRCV_43_AE_SID_CHECK_WAKEUP

```
#define CANTRCV_43_AE_SID_CHECK_WAKEUP
```

Service ID of CanTrcv_43_AE_CheckWakeup.

Definition at line 183 of file CanTrcv_43_AE.h.

6.1.3.17 CANTRCV_43_AE_SID_CHECK_WAKE_FLAG

```
#define CANTRCV_43_AE_SID_CHECK_WAKE_FLAG
```

Service ID of CanTrcv_43_AE_CheckWakeFlag.

Definition at line 185 of file CanTrcv_43_AE.h.

6.1.3.18 CANTRCV_43_AE_SID_DEINIT

```
#define CANTRCV_43_AE_SID_DEINIT
```

Service ID of CanTrcv_43_AE_DeInit.

Definition at line 187 of file CanTrcv_43_AE.h.

6.1.3.19 CANTRCV_43_AE_SID_MAINFUNCTION

```
#define CANTRCV_43_AE_SID_MAINFUNCTION
```

Service ID of CanTrcv_43_AE_MainFunction.

Definition at line 189 of file CanTrcv_43_AE.h.

6.1.3.20 CANTRCV_43_AE_SID_MAINFUNCTION_DIAGNOSTICS

```
#define CANTRCV_43_AE_SID_MAINFUNCTION_DIAGNOSTICS
```

Service ID of CanTrcv_43_AE_MainFunctionDiagnostics.

Definition at line 191 of file CanTrcv_43_AE.h.

6.1.4 Enum Reference

6.1.4.1 CanTrcv_43_AE_eDriverStatusType

```
enum CanTrcv_43_AE_eDriverStatusType
```

CanTrcv_43_AE_eDriverStatusType.

Driver status helps to prevent double driver initialization.

Enumerator

CANTRCV_43_AE_NOT_ACTIVE	Driver not initialized
CANTRCV_43_AE_ACTIVE	Driver ready

Definition at line 201 of file CanTrcv_43_AE.h.

6.1.5 Function Reference

6.1.5.1 CanTrcv_43_AE_GetVersionInfo()

```
void CanTrcv_43_AE_GetVersionInfo (
            Std_VersionInfoType * versioninfo )
```

CAN transceiver driver get version info function. SID is 0x04.

Returns the version information of this module.

Parameters

out	*versioninfo*	Pointer to where to store the version information of this module.

Returns

void

6.1.5.2 CanTrcv_43_AE_Init()

```
void CanTrcv_43_AE_Init (
            const CanTrcv_43_AE_ConfigType * ConfigPtr )
```

Initializes CanTrcv module. SID 0x00.

Initializes all transceivers configured in ConfigPtr parameter. The CANTRCV module shall be initialized by CanTrcv_43_AE_Init() service call during the start-up.

Parameters

in	*ConfigPtr*	Pointer to driver configuration structure.

Returns

void

Precondition

CanTrcv_43_AE_Init shall be called at most once during runtime.

Postcondition

CanTrcv_43_AE_Init shall initialize all the transceivers and set the driver in READY state.

6.1.5.3 CanTrcv_43_AE_SetOpMode()

```
Std_ReturnType CanTrcv_43_AE_SetOpMode (
            uint8 Transceiver,
            CanTrcv_TrcvModeType OpMode )
```

Sets the mode of the Transceiver to the value OpMode. SID 0x01.

Puts the device in one of following modes: normal, standby, sleep.

Parameters

in	*Transceiver*	Index of the transceiver.
out	*OpMode*	Desired operating mode.

Returns

Std_ReturnType Result of the transition.

Return values

E_OK	Operation executed successfully.
E_NOT_OK	Operation failed.

Precondition

CanTrcv module should be initialized before calling the CanTrcv_43_AE_SetOpMode method.

Postcondition

CanTrcv_43_AE_SetOpMode shall set the transceiver in the desired state.

6.1.5.4 CanTrcv_43_AE_GetOpMode()

```
Std_ReturnType CanTrcv_43_AE_GetOpMode (
            uint8 Transceiver,
            CanTrcv_TrcvModeType * OpMode )
```

Gets current operation mode. SID 0x02.

Gets the mode of the Transceiver and returns it in OpMode. The device is in one of following modes: normal, standby, sleep.

Parameters

in	*Transceiver*	CAN transceiver ID.
out	*OpMode*	Current operating mode.

Returns

Std_ReturnType Result of the transition.

Return values

E_OK	Operation executed successfully.
E_NOT_OK	Operation failed.

Precondition

CanTrcv module should be initialized before calling the CanTrcv_43_AE_GetOpMode method.

Postcondition

CanTrcv_43_AE_GetOpMode shall return the currently working mode of the transceiver.

6.1.5.5 CanTrcv_43_AE_GetBusWuReason()

```
Std_ReturnType CanTrcv_43_AE_GetBusWuReason (
            uint8 Transceiver,
            CanTrcv_TrcvWakeupReasonType * reason )
```

Gets the wakeup reason of the Transceiver and returns it in parameter Reason. SID 0x03.

The device can be woken up by Wake Up Pattern

Parameters

in	*Transceiver*	CAN transceiver to which API call has to be applied.
out	*Reason*	Pointer to wake up reason.

Returns

Std_ReturnType Result of the transition.

Return values

E_OK	Operation executed successfully.
E_NOT_OK	Operation failed.

Precondition

CanTrcv module should be initialized before calling the CanTrcv_43_AE_GetBusWuReason method.

Postcondition

CanTrcv_43_AE_GetBusWuReason shall return wake up reason.

6.1.5.6 CanTrcv_43_AE_SetWakeupMode()

```
Std_ReturnType CanTrcv_43_AE_SetWakeupMode (
            uint8 Transceiver,
            CanTrcv_TrcvWakeupModeType TrcvWakeupMode )
```

Enables, disables or clears wake up events of the Transceiver according to TrcvWakeupMode. SID 0x05.

Enables, disables or clears wake up functionality. If WU mode is disabled all wake up sources and interrupts are off. If WU mode is enabled, all wake up sources and interrupts are set according to configuration. If WU mode is clear, pending wake up flag is cleared.

Parameters

in	*Transceiver*	CAN Transceiver ID.
in	*TrcvWakeupMode*	Mode of wake up functionality (enabled, disabled, cleared).

Returns

Std_ReturnType Result of the transition.

Return values

E_OK	Operation executed successfully.

Return values

E_NOT_OK	Operation failed.

Precondition

CanTrcv module should be initialized before calling the CanTrcv_43_AE_SetWakeupMode method.

Postcondition

CanTrcv_43_AE_SetWakeupMode shall return status of the transceiver.

6.1.5.7 CanTrcv_43_AE_CheckWakeup()

```
Std_ReturnType CanTrcv_43_AE_CheckWakeup (
            uint8 Transceiver )
```

Service is called by underlying CANIF in case a wake up interrupt is detected. SID 0x07.

Reads wake up source from the device and reports it to ECUM.

Parameters

in	*Transceiver*	CAN transceiver ID.

Returns

Std_ReturnType Result of the transition.

Return values

E_OK	Operation executed successfully.
E_NOT_OK	Operation failed.

Precondition

CanTrcv module should be initialized before calling the CanTrcv_43_AE_CheckWakeup method.

Postcondition

CanTrcv_43_AE_CheckWakeup shall read and report wake up reason.

6.1.5.8 CanTrcv_43_AE_CheckWakeFlag()

```
Std_ReturnType CanTrcv_43_AE_CheckWakeFlag (
            uint8 Transceiver )
```

Requests to check the status of the wakeup flag from the transceiver hardware. SID 0x0E.

Checks wake up event and if WU occurred, reports it.

Parameters

in	*Transceiver*	CAN transceiver ID.

Returns

Std_ReturnType Result of the transition.

Return values

E_OK	Operation executed successfully.
E_NOT_OK	Operation failed.

Precondition

CanTrcv module should be initialized before calling the CanTrcv_43_AE_CheckWakeFlag method.

Postcondition

CanTrcv_43_AE_CheckWakeFlag shall check for wake up event, and if such event occurred, report it.

6.1.5.9 CanTrcv_43_AE_DeInit()

```
void CanTrcv_43_AE_DeInit (
            void  )
```

De-initializes the CanTrcv module. SID 0x10.

De-initialize all the transceivers. The CANTRCV module shall be de-initialized by CanTrcv_43_AE_DeInit() service call. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

Parameters

in	*None*	

Returns

void

Precondition

Before transceiver de-initalization, the driver must be initialized and the transceivers must not be in Start state.

Postcondition

CanTrcv_43_AE_DeInit shall de-initialize all the transceivers and set the driver in UNINIT state.

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.
In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.
Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.
Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.
NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at http://www.nxp.com/profile/terms, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.
Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.
NXP has a Product Security Incident Response Team (PSIRT) (reachable at mailto:PSIRT@nxp.com) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.